

Introduction au langage Python

Pr A. OUSAADANE

UMI, ENS

16/04/2026



Plan de cours

- 1 Introduction
- 2 Éléments de base
- 3 Structures de contrôle
- 4 Structures de données
- 5 Fonctions

- 1 Introduction
- 2 Éléments de base
- 3 Structures de contrôle
- 4 Structures de données
- 5 Fonctions

Qu'est-ce que Python ?

Définition

- Python est un langage de programmation **interprété** et de **haut niveau**.
- Il est connu pour sa simplicité et sa lisibilité grâce à une syntaxe claire et concise.
- Python est un langage polyvalent utilisé dans divers domaines :
 - Développement web (Django, Flask),
 - Science des données et intelligence artificielle,
 - Automatisation des tâches (scripts),
 - Calcul scientifique (NumPy, SciPy).
- Python est open-source et bénéficie d'une grande communauté de développeurs.

Pourquoi choisir Python ?

Atouts majeurs

- **Facilité d'apprentissage** : Une syntaxe simple et un faible coût d'entrée.
- **Large écosystème de bibliothèques** :
 - Calcul scientifique (SciPy, SymPy)
 - Traitement de données (Pandas, NumPy)
 - Apprentissage automatique (TensorFlow, scikit-learn)
 - Développement web (Flask, Django)
- **Portabilité** : Compatible avec Windows, Linux, macOS.
- **Communauté active** : Nombreuses ressources et forums d'entraide.
- **Utilisé par les grandes entreprises** : Google, NASA, META, etc.

Points clés

- **Langage interprété** : Exécution ligne par ligne, facile à déboguer.
- **Typage dynamique** : Pas besoin de déclarer le type des variables.
- **Multi-paradigme** : Supporte la programmation impérative, fonctionnelle et orientée objet.
 - **Programmation impérative** : Les instructions sont exécutées dans un ordre séquentiel, comme dans la plupart des langages classiques. Ex : utilisation de boucles et de conditions.
 - **Programmation fonctionnelle** : Les fonctions sont traitées comme des objets de première classe, permettant de passer des fonctions en arguments et de créer des fonctions anonymes (lambda).
 - **Programmation orientée objet (POO)** : Organiser le code en classes et objets. Ex : Encapsulation, héritage, polymorphisme.
- **Facilité d'intégration** : Peut être couplé avec C, C++, Java, Fortran.

Anaconda (recommandé)

- Distribution Python **tout-en-un** pour la science des données
- Inclut : Python, Jupyter Notebook, Spyder, NumPy, SciPy, Matplotlib, SymPy, etc.
- Téléchargement : <https://www.anaconda.com/download>

Jupyter Notebook

- Application web **interactive** mêlant code, texte, équations LaTeX et graphiques
- Lancement : `jupyter notebook` dans un terminal (ou via Anaconda Navigator)

Tout au long du module, nous travaillerons avec Jupyter Notebook.

- 1 Introduction
- 2 Éléments de base**
- 3 Structures de contrôle
- 4 Structures de données
- 5 Fonctions

Définition

- **Variables** : Nom symbolique pour stocker une valeur.
- **Types principaux** :
 - `int` : entiers (ex : 3, -5, 42)
 - `float` : décimaux (ex : 3.14, -2.7)
 - `str` : chaînes de caractères (ex : "Bonjour")
 - `bool` : booléens (True ou False)

Exemple

```
x = 5
pi = 3.14
nom = "Ali"
est_etudiant = True
```

Fonctions essentielles

- `print()` : Affiche des informations sur la console.
- `input()` : Récupère une saisie utilisateur (toujours sous forme de chaîne).

Exemple

```
nom = "Ali"  
print(f"Bonjour, {nom} !")  
  
age = int(input("Entrez votre âge : "))  
print("Vous avez", age, "ans.")
```

Opérations de base

Opérations arithmétiques

- Addition (+), Soustraction (-), Multiplication (*)
- Division (/), Division entière (//), Modulo (%), Puissance (**)

Opérations de comparaison

- Égalité (==), Différence (!=)
- Supérieur (>), Inférieur (<), etc.

Exemple

```
a = 10
b = 3
print("a + b =", a + b)
print("a % b =", a % b)
print(a == b) # False
print(b < a)  # True
```

- 1 Introduction
- 2 Éléments de base
- 3 Structures de contrôle**
- 4 Structures de données
- 5 Fonctions

Les conditions (if, elif, else)

Syntaxe

```
if condition1:  
    # exécuté si condition1 vraie  
elif condition2:  
    # exécuté si condition1 fausse et condition2 vraie  
else:  
    # exécuté si toutes les conditions sont fausses
```

Exemple : signe d'un nombre

```
nombre = int(input("Entrez un nombre : "))  
if nombre > 0:  
    print("Positif")  
elif nombre < 0:  
    print("Négatif")  
else:  
    print("Zéro")
```

Exemple : vérification de l'âge

```
age = int(input("Entrez votre âge : "))

if age >= 0:
    if age < 13:
        print("Enfant.")
    elif age < 18:
        print("Adolescent.")
    elif age < 65:
        print("Adulte.")
    else:
        print("Personne âgée.")
else:
    print("Âge invalide.")
```

La boucle for

Utilité

Répète un bloc un nombre déterminé de fois ou pour chaque élément d'une séquence.

Exemple : itération sur une liste

```
nombres = [1, 2, 3, 4, 5]
for nombre in nombres:
    print("Nombre :", nombre)
```

Fonction range()

```
for i in range(5):          # 0, 1, 2, 3, 4
    print(i)
for i in range(2, 8):       # 2, 3, 4, 5, 6, 7
    print(i)
```

La boucle while

Utilité

Répète un bloc tant qu'une condition est vraie (nombre d'itérations inconnu a priori).

Exemple : compteur

```
compteur = 1
while compteur <= 5:
    print("Compteur :", compteur)
    compteur += 1
```


Mots-clés break et continue

Définition

- `break` : Interrompt complètement la boucle.
- `continue` : Passe à l'itération suivante.

Exemple

```
for i in range(1, 10):  
    if i % 2 == 0:  
        continue          # Saute les nombres pairs  
    print(i)               # Affiche les impairs  
    if i > 5:  
        break             # S'arrête après 7  
# Résultat : 1, 3, 5, 7
```

- 1 Introduction
- 2 Éléments de base
- 3 Structures de contrôle
- 4 Structures de données**
- 5 Fonctions

Définition

- Structure **ordonnée** et **modifiable**.
- Peut contenir des éléments de types variés.

Exemple

```
nombres = [1, 2, 3, 4]
noms = ["Ali", "Ahmed", "Fatima"]
mixte = [1, "pi", 3.14]

print(nombres[0])    # 1 (premier élément)
print(nombres[-1])   # 4 (dernier élément)
```

Méthodes essentielles

- `append(x)` : Ajoute `x` à la fin
- `remove(x)` : Supprime la première occurrence de `x`
- `pop(i)` : Supprime et renvoie l'élément à l'indice `i`
- `sort()` : Trie la liste par ordre **croissant** (alphabétique pour les chaînes) et `sort(reverse=True)` pour un tri décroissant
- `len(liste)` : Longueur de la liste

Exemple

```
nombres = [1, 2, 3, 4]
nombres.append(5)      # [1, 2, 3, 4, 5]
nombres.remove(3)      # [1, 2, 4, 5]
print(len(nombres))    # 4
dernier = nombres.pop()
print(dernier)         # 5
```

Définition

- Structure **ordonnée** comme les listes, mais **immuable** (non modifiable après création).
- Syntaxe : utilisation de parenthèses () au lieu des crochets [].

Comparaison avec les listes

```
# Avec une liste (modifiable)
liste = [3, 4]
liste[0] = 10           # OK
print(liste)           # [10, 4]
```

```
# Avec un tuple (immuable)
point = (3, 4)
print(point[0])         # 3
point[0] = 10           # ERREUR ! Impossible de modifier
```

Définition

- Structure non ordonnée basée sur des paires **clé** → **valeur**.
- Recherche rapide par clé.

Exemple

```
personne = {"nom": "Ali", "age": 25}
print(personne["nom"])      # "Ali"
personne["age"] = 26        # Modification
personne["ville"] = "Rabat" # Ajout
```

Opérations sur les dictionnaires

Méthodes utiles

- `get(clé, défaut)` : Valeur associée ou valeur par défaut
- `keys()` : Liste des clés
- `values()` : Liste des valeurs
- `items()` : Paires (clé, valeur)

Exemple

```
personne = {"nom": "Ali", "age": 25}
print(personne.get("nom", "Inconnu"))
print(personne.keys())      # dict_keys(['nom', 'age'])
print(personne.values())    # dict_values(['Ali', 25])

for cle, val in personne.items():
    print(f"{cle}: {val}")
```

Les ensembles (set)

Définition

- Collection **non ordonnée** d'éléments **uniques**.
- Correspond à la notion mathématique d'ensemble.

Exemple : ensemble des diviseurs

```
diviseurs_12 = {1, 2, 3, 4, 6, 12}
diviseurs_18 = {1, 2, 3, 6, 9, 18}

# Diviseurs communs (intersection)
communs = diviseurs_12 & diviseurs_18
print(communs)  # {1, 2, 3, 6}

# Union des diviseurs
tous = diviseurs_12 | diviseurs_18
print(tous)     # {1, 2, 3, 4, 6, 9, 12, 18}
```

Les opérateurs `&` (intersection) et `|` (union) sont aussi disponibles.

Opérations sur les ensembles

Opérations et notations équivalentes

Opération	Méthode	Opérateur
Union	<code>A.union(B)</code>	$A \cup B$
Intersection	<code>A.intersection(B)</code>	$A \cap B$
Différence	<code>A.difference(B)</code>	$A - B$
Inclusion	<code>A.issubset(B)</code>	$A \subseteq B$

Exemple : ensembles de solutions

Solutions de $x^2 - 3x + 2 = 0$

`S1 = {1, 2}`

Solutions de $x^2 - 5x + 6 = 0$

`S2 = {2, 3}`

`print(S1 | S2)` # {1, 2, 3} (union)

`print(S1 & S2)` # {2} (solution commune)

- 1 Introduction
- 2 Éléments de base
- 3 Structures de contrôle
- 4 Structures de données
- 5 Fonctions**

Définition d'une fonction

Définition

- Les fonctions permettent de regrouper du code réutilisable.
- Les fonctions peuvent avoir des paramètres et renvoyer des valeurs.

Syntaxe

```
def nom_fonction(parametres):  
    # Corps de la fonction  
    return valeur
```

Exemple : racine cubique

```
def racine_cubique(x):  
    return x ** (1/3)  
  
print(racine_cubique(27))    # 3.0  
print(racine_cubique(8))    # 2.0
```

Paramètres par défaut

Définition

Des valeurs par défaut peuvent être assignées aux paramètres. Elles sont utilisées si l'argument n'est pas fourni lors de l'appel.

Exemple : racine cubique avec paramètre par défaut

```
def racine_cubique(x, precision=3):  
    return round(x ** (1/3), precision)  
  
print(racine_cubique(26))           # 2.962 (arrondi à 3 décimales)  
print(racine_cubique(26, 5))        # 2.9625 (arrondi à 5 décimales)  
print(racine_cubique(2))            # 1.26  
print(racine_cubique(2, 6))         # 1.259921
```

Variables locales et globales

- **Locale** : Déclarée à l'intérieur d'une fonction, inaccessible dehors.
- **Globale** : Déclarée en dehors de toute fonction, accessible partout.

Exemple

```
x = 5  # variable globale

def fonction():
    x = 10  # variable locale
    print(x)

fonction()  # Affiche 10
print(x)   # Affiche 5
```

Fonctions lambda

Définition

Fonctions **anonymes** sur une seule ligne.

Syntaxe :

```
lambda arguments: expression
```

Exemple

```
carre = lambda x: x ** 2  
print(carre(5))      # 25
```

```
addition = lambda x, y: x + y  
print(addition(3, 4)) # 7
```

```
nombres = [1, 2, 3, 4]  
# Applique lambda à chaque élément  
carres = list(map(lambda x: x**2, nombres))  
print(carres) # [1, 4, 9, 16]
```

Arguments variables : *args

Définition

- *args permet de passer un **nombre variable d'arguments positionnels** à une fonction.
- Les arguments sont automatiquement regroupés dans un **tuple**.
- Utile quand on ne connaît pas à l'avance le nombre d'arguments.

Exemple : somme de plusieurs nombres

```
def somme(*nombres):  
    # 'nombres' est un tuple contenant tous les arguments  
    return sum(nombres)  
  
print(somme(1, 2, 3))           # 6  
print(somme(5, 10, 15, 20))    # 50  
print(somme(7))                 # 7  
print(somme())                  # 0 (tuple vide)
```

Arguments variables : ****kwargs**

Définition

- ****kwargs** permet de passer un **nombre variable** d'arguments **nommés** (clé-valeur).
- Les arguments sont automatiquement regroupés dans un **dictionnaire**.

Exemple : fonction affine $f(x) = a \cdot x + b$

```
def affine(x, **params):  
    # 'params' est un dictionnaire  
    a = params.get("a", 1)    # défaut = 1 si "a" absent  
    b = params.get("b", 0)    # défaut = 0 si "b" absent  
    return a * x + b  
  
print(affine(5, a=2, b=3))    # 2*5 + 3 = 13  
print(affine(5, a=2))        # 2*5 + 0 = 10  
print(affine(5))              # 1*5 + 0 = 5  
print(affine(5, b=3))        # 1*5 + 3 = 8
```


À faire

- Utiliser des noms de variables **explicites** (ex : somme plutôt que s)
- Commenter les parties complexes du code
- Indenter correctement (4 espaces par niveau)

À éviter

- Les noms trop courts ou ambigus (ex : x, tmp)
- Mélanger espaces et tabulations pour l'indentation
- Les variables globales inutiles

Ce que nous avons vu dans ce chapitre

Acquis

- Installer Anaconda et utiliser Jupyter Notebook
- Manipuler les types de base (int, float, str, bool)
- Utiliser les structures de contrôle (if, for, while)
- Manipuler les structures de données (listes, tuples, dictionnaires, ensembles)
- Définir et appeler des fonctions (paramètres, lambda, *args, **kwargs)
- Connaître les bonnes pratiques de codage

**Prochaine séance : Bibliothèques fondamentales
(NumPy, SciPy, Matplotlib, SymPy)**